



19. Translation Lookaside Buffer

Review

TLB(Translation Lookaside Buffer)

TLB Basic Algorithms

Example: Accessing An Array

Locality

Temporal Locality

Spatial Locality

Who Handles The TLB MISS?

Hardware가 handle (CISC)

Software가 handle (RISC)

TLB Control Flow algorithm (OS Handled)

TLB entry

TLB Issue: Context Switching

To Solve Problem

Number of ASIDs (depends on architecture)

Another Case

When Is Page Sharing Useful?

1. Inter-Process Communication (IPC) - 프로세스 간 통신

2. Shared Libraries and Code Pages - 공유 라이브러리와 코드 페이지

3. Copy-on-Write (COW) after fork() - fork 후 Copy-on-Write

TLB Replacement Policy

LRU(Least Recently Used)

A Real TLB Entry

Coherence Bit (C)

Benefit

Dirty Bit (D)

TLB Entry vs Page Table Entry

TLB

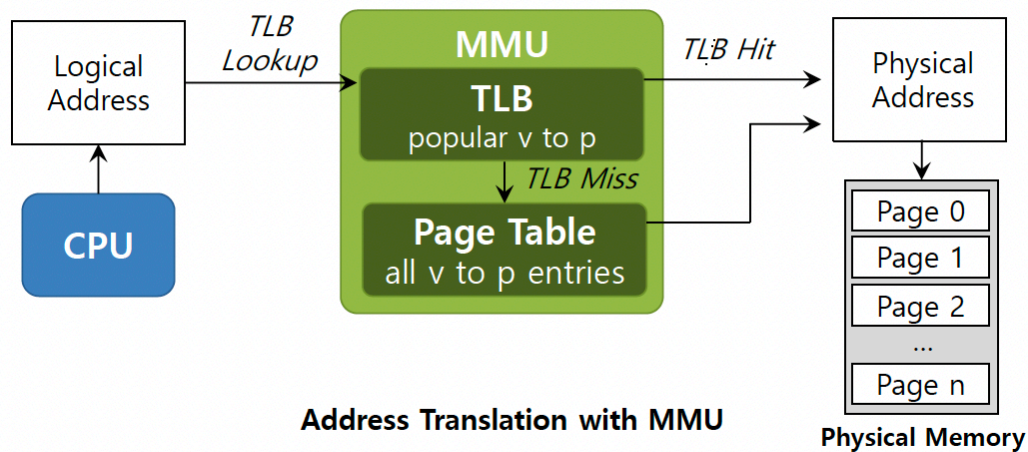
Example

Review

- paging이란 virtual address space를 일정한 크기(page)로 자르는 것 - VPN
- 물리적 메모리는 page frame 크기로 자름 - PFN
- page table은 mapping 해주는 역할을 함
- VA → PA를 하는 과정에서 page table을 참조하는데
 - 근데 이 Page Table은 어디에 있느냐? → OS 영역(DRAM 메모리)
 - ⇒ 성능 이슈
 - page table의 주소 값은 어떻게 아느냐
 - PTBR을 통해서
 - 시간 줄이기는?
 - TLB를 통해서

TLB(Translation Lookaside Buffer)

- MMU의 일부
- PTE cache다?
- A hardware cache of popular virtual-to-physical address translation
 - 예시)
 - $VA \leftarrow 32\text{bit}, 2^{32} = 4\text{G}$
 - page가 4KB라면, $4 * (2^{30}) / 4 * (2^{10}) = 2^{20} = 1\text{M}$
 - PTE가 각각 4 byte 크기라면, page table 사이즈가 $4 * 1\text{MB} = 4\text{MB} \Rightarrow \text{MMU에 들어가기에 너무 크다!}$
 - 보통 TLB는 32, 64, 128개의 entry 가짐



- 띠용? page Table이 MMU 안에 있는거 아님. 그림 잘못된 거
 - 실제로는 TLB만 MMU 안에
 - 영재킴 피셜) PTBR도 MMU안에 있다고는 하는데 실제로는 아닌건지 모르겠네

CPU Core

- ├ 레지스터들 (범용 레지스터, PC, SP 등)
- ├ CR3 레지스터 (PTBR)
- └ MMU (메모리 관리 유닛)
 - ├ TLB (주소 변환 캐시)
 - ├ 주소 변환 로직
 - └ 페이지 테이블 워커

TLB Basic Algorithms

SHIFT는 12?

```

VPN = (VirtualAddress & VPN_MASK) >> SHIFT
(Success, TlbEntry) = TLB_Lookup(VPN) // O(1)에 가능하다 병렬적으로
if (Success == TRUE){ // TLB Hit
    if(CanAccess(TlbEntry.ProtectBit) == True){
        offset = VirtualAddress & OFFSET_MASK
        PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
        AccessMemory(PhysAddr)
    }
    else RaiseException(PROTECTION_ERROR)
}

```

```

else { // TLB Miss
    PTEAddr = PTBR + (VPN * sizeof(PTE))
    PTE = AccessMemory(PTEAddr)
    if(PTE.Valid == False)
        RaiseException(SEGFAULT) ;
    else {
        TLB_Insert( VPN , PTE.PFN , PTE.ProtectBits)
        RetryInstruction()
    }
}
}

```

- **(11-12 lines)** The hardware accesses the page table to find the translation.
- **(16 lines)** updates the TLB with the translation.

Example: Accessing An Array

	OFFSET				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

```

0:  int sum = 0 ;
1:  for( i=0; i<10; i++){
2:      sum+=a[i] ;
3:  }

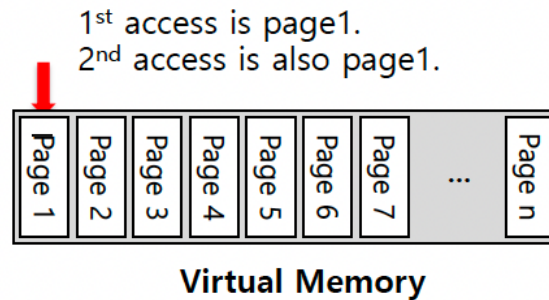
```

- miss, hit, hit
- miss, hit, hit, hit
- miss, hit, hit
- **The TLB improves performance due to spatial locality**
 - 3 misses and 7 hits. Thus TLB hit rate is 70%.

Locality

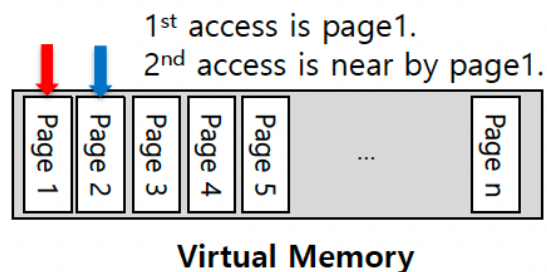
Temporal Locality

- An instruction or data item that has been recently accessed will likely be re-accessed soon in the future.



Spatial Locality

- If a program accesses memory at address x, it will likely soon access memory near x.



Who Handles The TLB MISS?

Hardware가 handle (CISC)

- The hardware has to know exactly where the page tables are located in memory
하드웨어는 memory 상에서 페이지 테이블 위치를 일단 알아야함

- The hardware would "walk" the page table, find the correct page-table entry and **extract** the desired translation, **update** and **retry** instruction.
하드웨어가 페이지 테이블을 "**걸어다니면서(walk)**" 올바른 엔트리를 찾아, 찾은 변환 정보를 추출하고, TLB에 **업데이트**하고 명령어 **재시도**
- **hardware-managed TLB.**
- **Intel x86**

Software가 handle (RISC)

- On a TLB miss, the hardware raises exception (trap handler).
 - **Trap handler is code** within the OS that is written with the express purpose of handling TLB miss

TLB Control Flow algorithm (OS Handled)

- 한번 보세요?

```

VPN = (VirtualAddress & VPN_MASK) >> SHIFT
(Success, TlbEntry) = TLB_Lookup(VPN)
if(Success == True) // TLB Hit
  if(CanAccess(TlbEntry.ProtectBits) == True)
    Offset = VirtualAddress & OFFSET_MASK
    PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
    Register = AccessMemory(PhysAddr)
  else
    RaiseException(PROTECTION_FAULT)
else// TLB Miss
  RaiseException(TLB_MISS)

```

TLB entry

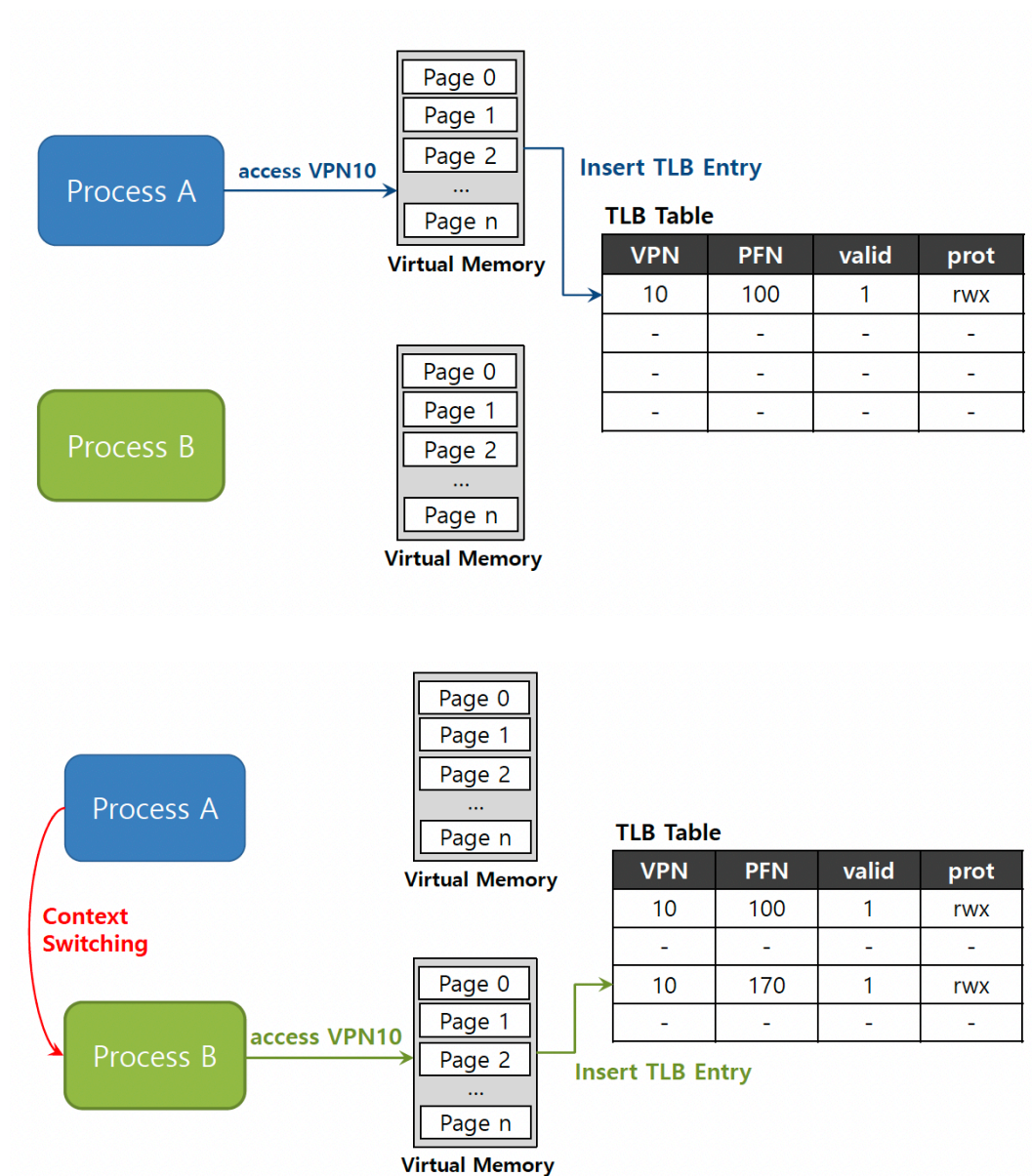
- TLB is managed by **Full Associative** method.
아무데나 들어갈 수 있다!
 - A typical TLB has 32, 64, or 128 entries.

VPN	PFN	other bits
-----	-----	------------

- Hardware searches the entire TLB in parallel to find the desired translation.
- other bits: **valid bits, protection bits, address-space identifier, dirty bit**

+) direct mapped cache면 PT 값에 따라 TLB에 들어갈 수 있는 위치가 정해지게 됨

TLB Issue: Context Switching



| 어떤 process의 entry인지 구분할 수 없다는 문제점 발생

To Solve Problem

- Provide an **address space identifier(ASID)** field in the TLB

TLB Table

VPN	PFN	valid	prot	ASID
10	100	1	rwX	1
-	-	-	-	-
10	170	1	rwX	2
-	-	-	-	-

Number of ASIDs (depends on architecture)

- ASIDs enable TLB entries from multiple processes to coexist, reducing the need for full TLB flushes during context switches.
ASID가 있어서 여러 process가 공존할 수 있다!! 고맙다~ ASID 민수야! & context switch 때마다 TLB flush 필요성 줄임
- The ASID count is architecture-specific (not standardized).

Architecture	ASID Bit Width	Possible ASID Count
x86 (Intel/AMD)	Typically 12 bits	$2^{12} = 4096$ ASIDs
ARMv8 (AArch64)	Usually 8 bits	$2^8 = 256$ ASIDs
RISC-V (Sv39/Sv48)	Implementation-dependent, often 16 bits	up to $2^{16} = 65,536$ ASIDs

- **문제:** ASID가 부족하면?
 - OS가 ASID 재사용 (오래된 프로세스 ID를 새 프로세스에게로)
- **위험:** stale TLB entries → wrong translation
 - 오래된 TLB 엔트리가 남아있으면 **잘못된 주소 변환** 발생!
 - 예전 프로세스의 ASID=5 엔트리가 TLB에 남아있는데
 - 새 프로세스한테 ASID=5를 쥐버리면 엉뚱한 물리 주소로 접근할 수 있어!
- **해결:** flush TLB before ASID reuse

- ASID 재사용하기 전에 **해당 ASID의 TLB 엔트리를 전부 flush**해
- OS invalidates all TLB entries tied to the reused ASID to ensure correctness

Another Case

- Two processes share a page.
 - Process 1 is sharing physical page 101 with Process2.
 - P1 maps this page into the 10th page of its address space.
 - P2 maps this page to the 50th page of its address space.

VPN	PFN	valid	prot	ASID
10	101	1	rwX	1
-	-	-	-	-
50	101	1	rwX	2
-	-	-	-	-

Sharing of pages is useful as it reduces the number of physical pages in use.

When Is Page Sharing Useful?

1. Inter-Process Communication (IPC) - 프로세스 간 통신

- Shared memory regions (e.g., mmap, shm_open, CreateFileMapping) let processes exchange data efficiently.
해석: 공유 메모리 영역 만들어서 데이터 주고 받기 - ex) 함수들: mmap, shm_open, CreateFileMapping
- Both processes see the same physical page, so writes by one are immediately visible to the other.
해석: 두 프로세스가 같은 물리 페이지 봐 - 프로세스 A가 쓴 내용을 프로세스 B가 즉시 볼 수 있어
- Example: a database server and a client sharing a memory segment for high-speed data transfer.
해석: 예시) 데이터베이스 서버랑 클라이언트가 메모리 공유해서 고속으로 데이터 전송

2. Shared Libraries and Code Pages - 공유 라이브러리와 코드 페이지

- Multiple processes can map the same read-only code (e.g., libc) into their address spaces at different virtual addresses.

해석: 여러 프로세스가 같은 읽기 전용 코드 ex) libc를 사용할 때, 각자 다른 가상 주소에 매핑하지만, 물리적으로는 같은 페이지 씬

- Saves physical memory and reduces I/O overhead.

해석: I/O 오버헤드 감소 (디스크에서 한 번만 읽으면 됨)

3. Copy-on-Write (COW) after fork() - fork 후 Copy-on-Write

- After a fork(), the child process shares all physical pages with the parent until one modifies a page.

- Both initially point to the same page; the kernel duplicates the page only when a write occurs.

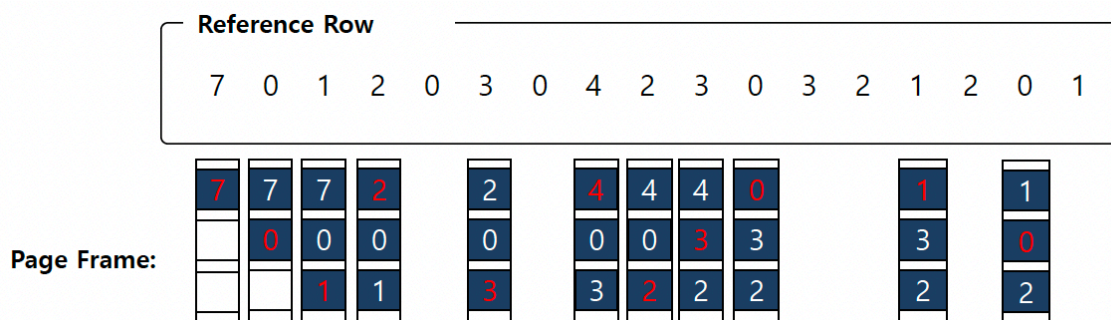
- 해석: fork하고 나서 처음에는 복사 안하고 같은 물리 페이지 공유, 둘 다 같은 페이지 가리킴

그런데 누가 쓰기를 하면 그제서야 kernel이 페이지 복사

TLB Replacement Policy

LRU(Least Recently Used)

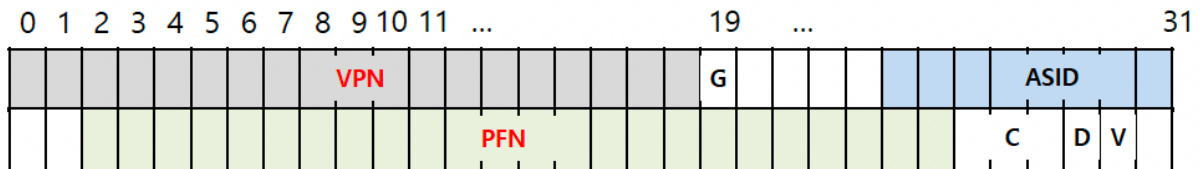
- Evict an entry that has not recently been used - 최근에 사용하지 않은것을 빼라
- Take advantage of **locality** in the memory-reference stream
- TOTAL: 11 TLB miss



- 7을 가장 오래 전에 썼으니까 빼라

- 근데 time stamp도 필요함 - 시간 알아야하니까
 - Heuristic 접근이 필요하다!

A Real TLB Entry



Flag	Content
19-bit VPN	The rest reserved for the kernel.
24-bit PFN	Systems can support with up to 64GB of main memory(pages).
Global bit (G)	Used for pages that are globally-shared among processes.
ASID	OS can use to distinguish between address spaces.
Coherence bit(C)	determine how a page is cached by the hardware.
Dirty bit(D)	marking when the page has been written.
Valid bit (V)	tells the hardware if there is a valid translation present in the entry.

Coherence Bit (C)

- TLB 엔트리가 현재 페이지 테이블과 동기화되어 있는지 표시
 - 페이지 테이블이 바뀌었는데 TLB가 아직 업데이트 안 됐으면?? → 문제!
- 언제 C = 0으로 바뀌냐?
- ex) page replacement(swapping) - SSD에 저장하는거라고?, page migration(for NUMA balancing) - 서버 컴에는 하나의 SoC에 다 집어 넣을 수 없다? socket?? 부하 불균형? 한 CPU만 busy? thread 재분배, copy를 하고 PFN이 바뀔?, Demand paging(page fault handling), Permission change, Kernel page remapping

Coherence Bit	Meaning	Behavior
---------------	---------	----------

1 (coherent/valid)	TLB 엔트리가 페이지 테이블과 일치함	사용 가능
0 (incoherent/invalid)	페이지 테이블 매핑이 바뀜	entry 무시, TLB miss 발생시켜 다시 로드

Benefit

- Instead of flushing the entire TLB, hardware can mark only affected entries as incoherent by clearing their coherence bits, ensuring (i) Fast selective invalidation, (ii) Correct address translation, (iii) Avoiding use of stale permissions

Dirty Bit (D)

- 페이지가 메모리에 로드된 후 **수정(write)됐는지 표시**
 - **D = 0** → 디스크 내용과 동일
 - **D = 1** → 디스크 내용과 다름
- 왜 필요한가?
 - 페이지를 메모리에서 disk 혹은 SSD로 쫓아낼 때 (evict / replace):
 - D = 0이면? 그냥 버려도 됨
 - D = 1이면? 디스크에 써야 함 - 수정되었으니까
- 언제 D = 1로 세팅되는가
 - 프로세스가 그 페이지에 **write 연산**을 할 때 **하드웨어가 자동으로** 세팅해줘 (MMU가 알아서)
- 언제 D = 0으로 클리어되는가
 1. 페이지를 디스크에 다시 쓴 후(write back 완료)
 2. 처음 페이지를 load하거나 다시 매핑할 때

TLB Entry vs Page Table Entry

- 둘 다 **주소 변환 정보**(가상 주소 → 물리 주소)를 저장 그러나 **목적**이 다르니까 저장하는 필드가 완전히 똑같지 않아!

TLB

- 하드웨어 캐시
- 그래서 속도 최적화를 위한 추가 필드들이 있어
 - 빠른 검색(lookup), 교체(replacement), 프로세스 격리 관리(process isolation)

Example

- TLB에만 있고 PTE에는 없는 field들

Field	Description
ASID (Address Space Identifier)	서로 다른 프로세스의 TLB 엔트리를 구분(x86에서는 PCID라고 부름)
Virtual Page Number (VPN)	검색과 매칭을 위해 내부적으로 사용 (어떤 가상 페이지인지 빠르게 찾기 위해)
Valid / LRU / Replacement metadata	entry validity and replacement policy를 위한 하드웨어 전용 비트들 Valid: 이 엔트리가 유효한지 LRU: 교체 정책용 (Least Recently Used) Replacement: 어떤 걸 쫓아낼지 결정
Tag / Cacheability hints	일부 CPU에 있는 필드 Tag: 빠른 검색용 Cacheability: 캐시 정책 힌트, 일관성(coherence) 관리

- The TLB adds hardware-specific metadata like ASID and lookup tags for speed and context isolation.
- The PTE adds OS-level metadata like Accessed, Dirty, and cache policy bits for page management and protection.

- page table은 entry에 VPN이 없다? → 인덱스가 vpn이여서?
- TLB는 있다?

왜 24bit PFN이면 64gb지?

TLB의 실제 물리적 크기와 다를 수 있다?

cache를 어떻게 구현할 것인가

1. fully associative cache → TLB
2. direct mapped cache
3. set associative cache → 2way or 4way, CPU cache(L1, L2)?

누가 무엇을 하는지 구분해라

TLB Issue: Context Switching

reuse할때 flush?

VPN이 같은 경우 ASID로 구분

PFN이 같은 경우